

# Aplicații pentru programe de tip *„Compute Shader”* - Sisteme de particule

Cristian Lambru: 2025-26







# Cuprins

1. Introducere
2. Studiu de caz - Jocul "*Cyberpunk 2077*"
3. Studiu de caz - efectul de *ninsoare*





# Introducere (I)

Sistemele de particule reprezintă o *tehnică de desenare în programarea graficii*, utilizată pentru simularea unor *fenomene dinamice*, ce *NU conțin o suprafață bine definită*. Câteva exemple de astfel de fenomene ar putea fi:

- Exploziile;
- Fumul;
- Focul.

Aceste fenomene/efecte ar fi *greu de desenat* prin metode clasice de programarea graficii, ce utilizează *desenarea unei geometrii poligonale*.

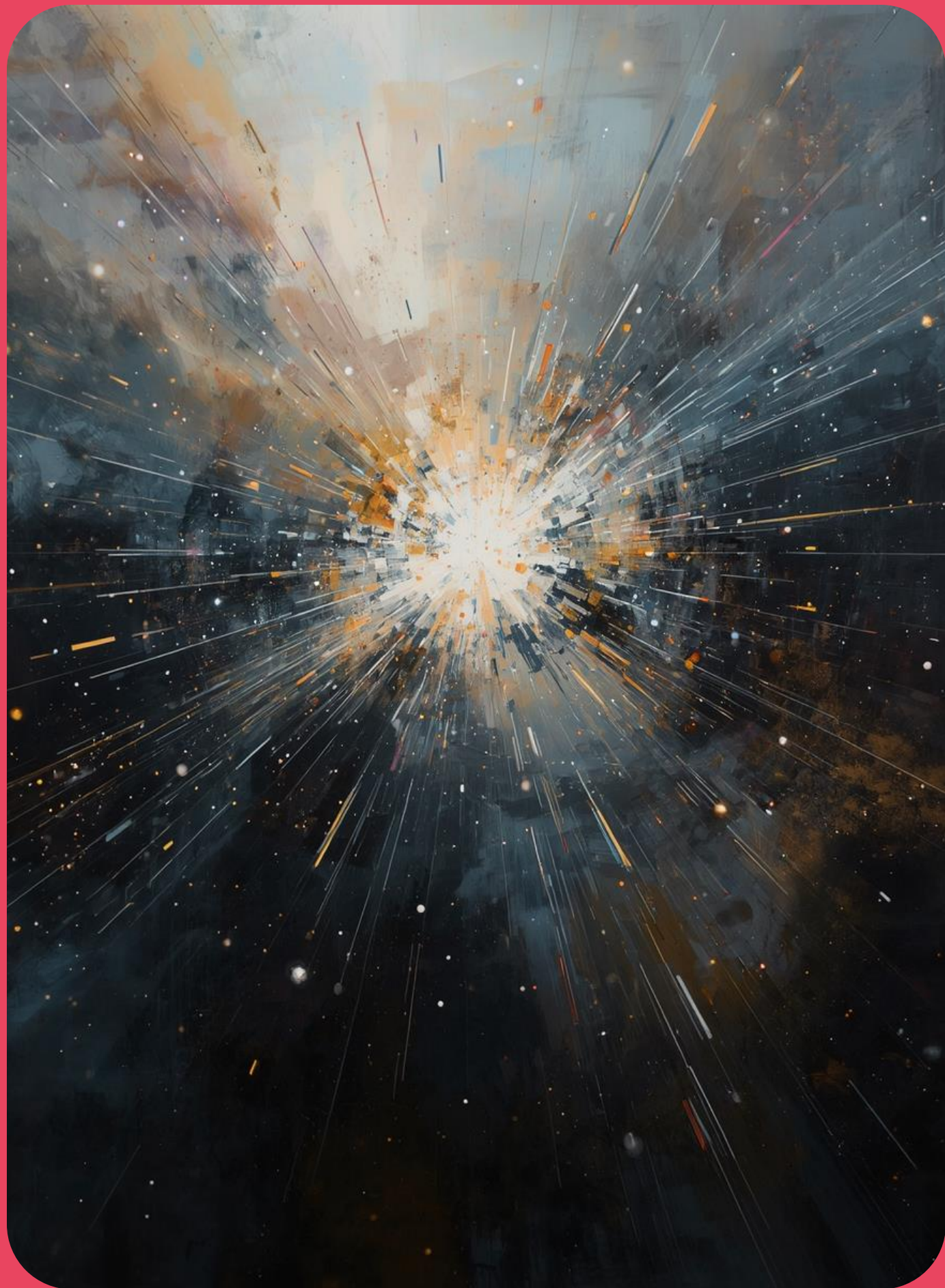


# Introdudere (II)

[https://www.youtube.com/watch?v=Tq\\_sSxDE32c](https://www.youtube.com/watch?v=Tq_sSxDE32c)

1 - W. T. Reeves. 1983. Particle Systems—a Technique for Modeling a Class of Fuzzy Objects. ACM Trans. Graph. 2, 2 (April 1983), 91–108. <https://doi.org/10.1145/357318.357320>





# Introducere (III)

Listă cuprinzătoare de exemple:

- Explozii;
- Fum;
- Foc;
- Apa in miscare;
- Scantei;
- Frunze care cad;
- Pietre care cad;
- Nori;
- Ceata;
- Ninsoare;
- Praf;
- Coada de meteoriti;
- Stele.

# Studiu de caz - Jocul „*Cyberpunk 2077*”

<https://www.youtube.com/watch?v=k6eld1Vjzuo>



# Studiu de caz - efectul de *ninsoare* (I)

<https://www.youtube.com/watch?v=RGo45ONOG3s>





# Studiu de caz - efectul de *ninsoare* (II)

Desenăm un set de patrulaterare orientate spre observator.

Patrulaterarele sunt desenate cu o imagine de fulg de zăpadă.



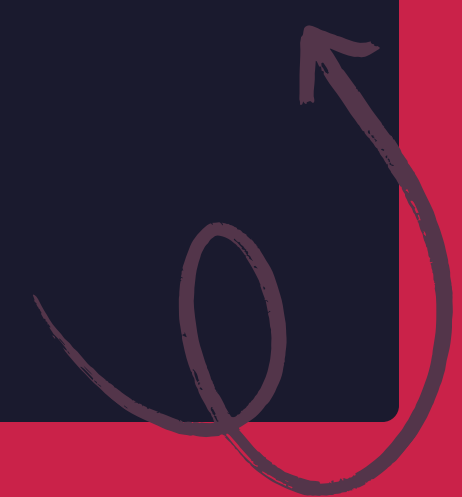


# Studiu de caz - efectul de *ninsoare* (III)

Stocăm coordonatele patruleterelor care reprezintă fulgii de zăpadă într-un obiect de tip „*Shader Storage Buffer Object*”.

```
struct Particle
{
    vec4 position;
    vec4 velocity;
};

layout(std430, binding = 0) buffer ParticleBuffer
{
    Particle particles[];
};
```



# Studiu de caz - efectul de *ninsoare* (IV)

Modificăm coordonatele la care se află fulgii de zăpadă în spațiul lumii pe baza unor reguli. Modificările sunt efectuate într-un program de tip „*Compute Shader*”.

```
#version 430

layout(local_size_x = 256) in;

struct Particle
{
    vec4 position;
    vec4 velocity;
};

layout(std430, binding = 0) buffer ParticleBuffer
{
    Particle particles[];
};
```





# Studiu de caz - efectul de *ninsoare* (V)

Modificăm coordonatele la care se află fulgii de zăpadă în spațiul lumii pe baza unor reguli. Modificările sunt efectuate într-un program de tip „*Compute Shader*”.

```
uniform float deltaTime;
uniform float fallAcceleration;

void main()
{
    uint id = gl_GlobalInvocationID.x;

    Particle p = particles[id];

    p.velocity.y -= fallAcceleration * deltaTime;
    p.position.xyz += p.velocity.xyz * deltaTime;

    particles[id] = p;
}
```



# Studiu de caz - efectul de *ninsoare* (VI)

Resetăm coordonata și viteza particulei în momentul în care aceasta a ajuns sub nivelul solului.

```
#version 430

layout(local_size_x = 256) in;

struct Particle
{
    vec4 position;
    vec4 velocity;
};

layout(std430, binding = 0) buffer ParticleBuffer
{
    Particle particles[];
};

uniform float deltaTime;
uniform float minY;
uniform float maxY;
uniform float areaSize;
uniform float fallAcceleration;

float rand(float n)
{
    return fract(sin(n) * 43758.5453123);
}
```





# Studiu de caz - efectul de *ninsoare* (VII)

Resetam coordonata si viteza particulei in momentul in care aceasta a ajuns sub nivelul solului.

```
void main()
{
    uint id = gl_GlobalInvocationID.x;

    Particle p = particles[id];

    p.velocity.y -= fallAcceleration * deltaTime;
    p.position.xyz += p.velocity.xyz * deltaTime;

    // reset particle
    if (p.position.y < minY) {
        float rx = rand(id * 12.9898f);
        float rz = rand(id * 78.233f);
        float ry = rand(id * 37.719f);

        p.position.xyz = vec3 (
            mix(-areaSize / 2.0f, areaSize / 2.0f, rx),
            maxY,
            mix(-areaSize / 2.0f, areaSize / 2.0f, rz)
        );

        p.velocity.xyz = vec3 (
            rx * 0.05f,
            ry - 1.0f,
            rz * 0.05f
        );
    }

    particles[id] = p;
}
```



# Studiu de caz - efectul de *ninsoare* (VIII)

Desenarea patrulaterului orientat spre observator se poate realiza după cum urmează.

```
#version 430

layout(location = 0) in vec2 quadVertex;
// valori: (-0.5,-0.5), (0.5,-0.5), (0.5,0.5), (-0.5,0.5)

struct Particle
{
    vec4 position;
    vec4 velocity;
};

layout(std430, binding = 0) readonly buffer ParticleBuffer
{
    Particle particles[];
};

uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;
uniform float quadSize;
```

1 - glDrawElementsInstanced - OpenGL 4 Reference Pages (2026) Khronos.org. Available at: <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstanced.xhtml> (Accessed: 29 April 2026).



# Studiu de caz - efectul de *ninsoare* (IX)

Desenarea patrulaterului orientat spre observator se poate realiza după cum urmează.

```
out vec2 tex_coords;

void main()
{
    uint particleID = gl_InstanceID;

    vec3 particleCenter = particles[particleID].position.xyz;

    vec3 cameraRightDirection = vec3(viewMatrix[0][0], viewMatrix[1][0], viewMatrix[2][0]);
    vec3 cameraUpDirection    = vec3(viewMatrix[0][1], viewMatrix[1][1], viewMatrix[2][1]);

    vec3 worldPos =
        particleCenter +
        cameraRightDirection * quadVertex.x * quadSize +
        cameraUpDirection   * quadVertex.y * quadSize;

    gl_Position = projectionMatrix * viewMatrix * vec4(worldPos, 1.0);

    tex_coords = quadVertex + vec2(0.5);
}
```

1 - glDrawElementsInstanced - OpenGL 4 Reference Pages (2026) Khronos.org. Available at: <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstanced.xhtml> (Accessed: 29 April 2026).

# Studiu de caz - efectul de *ninsoare* (X)

Efectul de *ninsoare* se poate îmbunătăți prin următoarele:

- Se introduce o traiectorie sinusoidală stânga-dreapta, astfel încât să se *simuleze frecarea fulgilor de zăpadă cu aerul*;
  - Se identifică corect *intersecția particulei cu toată geometria scenei*, nu doar cu solul;
  - Se introduce *redimensionare pe durata de viață a unei particule*: particula pornește cu o dimensiune maximă și la sfârșitul duratei de viață are o dimensiune minimă;
  - Se introduce un *efect de transparență pe durata de viață a unei particule*: particula pornește cu transparență minimă și la sfârșitul vieții are transparență maximă;
- 

# Studiu de caz - efectul de *ninsoare* (XI)

- Se poate utiliza o imagine ce contine o matrice de stări ale unui fulg de zăpadă și se afișează pe fața patrulaterului, pe rând, stările fulgului.

